EnergyGate: Defending Battery-Powered IoT Nodes Against Post-Quantum Handshake Exhaustion Attacks

Ramkiran Garuda
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
garuda.ramkiran2024@vitstudent.ac.in

Chaitanya Patil
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
chaitanya.patil2024@vitstudent.ac.in

Adithya Binoj Nair
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
adithya.nair2024@vitstudent.ac.in

***Abstract*—A battery-powered field device that accepts post-quantum handshakes pays for every request it is asked to check, whether or not the request is genuine. We measure that cost on commodity ESP32 microcontrollers running ML-KEM (FIPS 203) and ML-DSA (FIPS 204) over a broadcast ESP-NOW mesh, through a calibrated high-side INA219 rig. At NIST level 3 the responder's share of one handshake costs ≈14.9 mJ above idle; a forged HELLO carrying a structurally valid signature forces a full ML-DSA-65 verification (≈3.5 mJ) before it fails. Over 120 s attacks ($n = 5$, 95 % CI), an undefended insider raises the victim's power by $+91.2 \pm 3.3$ mW and cuts legitimate handshake success to 0.21; a single-signature forger adds $+19.0 \pm 1.4$ mW. We present *EnergyGate*, which decides at the first fragment of a HELLO—before reassembly and before any public-key work—whether to spend, challenge or drop, using an eight-feature link score and a joule token bucket. It removes the insider drain ($+1.1$ mW) without locking out the real gateway, but an attacker that echoes the broadcast cookie defeats both a cookie baseline and the gate's cookie rule ($+16.0$ mW). The budget is what holds: in 600 s echo attacks it empties and admissions settle at exactly the refill bound $r/c_h = 2.5\,\mathrm{s}^{-1}$, independent of the classifier. Retrained on recorded traces and evaluated by session, the gate's tree reaches AUC 0.95, yet slow-drip attackers remain invisible to it, which is why the bound, not the score, is the guarantee.**

***Index Terms*—post-quantum cryptography, ML-KEM, ML-DSA, energy exhaustion, denial of service, ESP32, ESP-NOW, admission control**

I. Introduction

Post-quantum key exchange is arriving on constrained devices. ML-KEM and ML-DSA [1], [2] run on a commodity ESP32 in milliseconds to tens of milliseconds per operation (Table II), so the question is no longer whether a microcontroller *can* run a signed post-quantum handshake, but what it costs to let anyone *ask* it to.

Field devices in distribution grids, pipelines and environmental monitoring often run on a battery and share a broadcast radio. A post-quantum handshake is an expensive operation, unauthenticated until verified, triggered by a message anyone in range can send. At NIST level 3 a single HELLO is 4517 bytes—20 ESP-NOW frames—and checking it means an ML-DSA-65 verification; admitting it commits the device to an encapsulation, a signature and a 19-frame response. The cryptography protects the message. Nothing in it protects the battery.

Our motivating scenario follows the 2015 attack on the Ukrainian grid [5], whose chain ran from phishing through malicious attachments to the field network and physical access. We study the field-network step: an attacker within radio range of a battery-powered node who wants to shorten its life or deny service without breaking any cryptography. Our contributions:

- **C1, measured cost.** Energy per post-quantum operation and per handshake on real ESP32 hardware, through a calibrated INA219 rig, at two parameter sets (Sections VI–VII).
- **C2, the attack.** Victim power and legitimate-handshake success for six attacker behaviours under four defences, with five repeats and 95 % CIs (Section VIII).
- **C3, a budget-bounded learned gate.** EnergyGate decides at the first fragment whether to spend, challenge or drop. We show on the rig that its joule bucket binds at the predicted rate even when the cookie and the score are both defeated (Sections V, VIII, IX).
- **C4, the artifact.** Firmware, rig description, calibration data, raw logs and scripts that rebuild every table.

II. Background and Related Work

ML-KEM and ML-DSA. ML-KEM is a module-lattice KEM and ML-DSA a module-lattice signature with rejection-sampled signing, so its signing time is variable [1], [2]. We use PQClean's portable C implementations [3].

ESP-NOW is Espressif's connectionless vendor-frame protocol, at most 250 bytes per frame in v1 [4]. We broadcast, so there is no link-layer retransmission and every station in range hears every frame.

PQC cost on small devices. Gillot *et al.* measure ML-KEM/ML-DSA/SLH-DSA energy on Raspberry Pi and laptop platforms [6] and TLS 1.3 PQC handshakes on Pico W, ESP32 and Pi Zero [7]; Karcz *et al.* time the NIST schemes on an

ESP32-C6 [8]. All characterise the cost the device *wants* to pay. None measures the cost an adversary chooses when to impose.

Handshake exhaustion. Lee *et al.* show PQC-TLS prolongs high server CPU load under handshake exhaustion by up to 88× and degrades learned IDSs [9]—server CPU, not battery. Hummen *et al.* protect constrained peers' public-key work with puzzles [10]; client puzzles and cookies for key exchange are well studied [11], [12]. On a broadcast medium a stateless cookie only stops blind spoofers.

Battery exhaustion. Stajano and Anderson named "sleep deprivation torture" [13]; Pirretti *et al.* analyse sleep-deprivation attacks in sensor networks [14]. Those defences sit at the MAC/clustering layer. From a first, non-systematic search (`paper/writing/related_work.md`) we found no prior measurement of the battery cost of abusive post-quantum handshake requests on a microcontroller and no defence that bounds it with a joule budget independent of a classifier; we do not claim this is exhaustive.

III. Threat Model

Setting. One battery-powered responder (*field-1*) and one mains-powered gateway (the initiator) share a broadcast ESP-NOW channel. Only the gateway initiates; a real field node never sends a HELLO.

Attacker. Within radio range; transmits arbitrary frames at any rate; holds no legitimate long-term key; spoofs any sender id (ESP-NOW frames are unauthenticated); hears every frame, so it can replay HELLOs and echo any broadcast cookie; and knows the defence, its features, thresholds and budget (Kerckhoffs). Behaviours:

- **FLOOD**: forged HELLOs under the gateway's id with random signature bytes. ML-DSA rejects them during signature unpacking (≈1.7 ms), so they are cheap to reject.
- **FORGE**: one *structurally valid* ML-DSA-65 signature from the attacker's own key, resent under the gateway's id with fresh (epoch, seq). It passes unpacking, so field-1 runs the full verification and fails only at the end. One attacker signature forces a full check on every resend.
- **INSIDER**: HELLOs validly signed with an *enrolled* key (a compromised peer); each admitted HELLO costs a full responder handshake.
- **SLOW_DRIP**: INSIDER at ≈1 HELLO/min, below any fixed rate limit.
- **ADAPTIVE / ADAPTIVE_SPOOF**: knows EnergyGate's features; keeps `hs_fail` and `dup_pct` at zero and paces `hs_per_s` under the tree's split with AIMD back-off on a heard CHALLENGE.
- **+ECHO**: any of the above, also echoing the cookie from each CHALLENGE it hears.

Out of scope: jamming, physical capture, side channels, breaking ML-KEM/ML-DSA. The attacker wins by maximising victim energy or denying the gateway a session; we report both.

TABLE I
Message sizes and ESP-NOW frames (235 B payload per frame)

Level	HELLO	RESPONSE	CONFIRM
1	3244 B, 14 fr.	3220 B, 14 fr.	48 B, 1 fr.
3	4517 B, 20 fr.	4429 B, 19 fr.	48 B, 1 fr.

IV. System: the Post-Quantum Handshake

A signed-KEM exchange in three messages, on PQClean (commit `0586a824`):

$$I \to R : \mathsf{HELLO} = \text{cookie} \,\|\, n_i \,\|\, pk_{\mathrm{KEM}} \,\|\, \sigma_I(\mathrm{th}_1)$$
$$R \to I : \mathsf{RESPONSE} = n_i \,\|\, n_r \,\|\, ct \,\|\, \sigma_R(\mathrm{th}_2, R, I)$$
$$I \to R : \mathsf{CONFIRM} = n_r \,\|\, \mathrm{tag}$$

with $\mathrm{th}_1 = \mathrm{XOF}(I, R, n_i, pk)$, $\mathrm{th}_2 = \mathrm{XOF}(\mathrm{th}_1, n_r, ct)$, session keys $\mathrm{XOF}(ss, \mathrm{th}_2)$ and SHAKE256 as XOF. Static provisioned ML-DSA keys authenticate; an ephemeral ML-KEM key agrees. Session keys drive AES-256-GCM. The cookie lies outside the signed transcript so that, after a CHALLENGE, the initiator resends the same signed HELLO without signing again. The protocol is host-tested against forged signatures, wrong keys, tampered fields and replays, but **not formally verified**.

Level 1 is ML-KEM-512 + ML-DSA-44; level 3 (default) is ML-KEM-768 + ML-DSA-65. Level 5 does not fit: ML-DSA-87 signing needs more stack than the largest free heap block (110 580 B). Table I gives message sizes.

Resource bounds. The reassembler tracks at most 8 streams with a 3 s timeout. All post-quantum work runs in one task with a reserved stack (90 112 B at level 3); while busy, admissions are refused (`busy_drop`), which caps the crypto rate. **Replay guard:** per sender, the last 8 verified epochs with their highest verified sequence number; unverified HELLOs never advance it.

V. EnergyGate

Placement. On field-1, whose battery is at risk. Inbound HELLO checks run: header sanity, replay/freshness, allow-list, EnergyGate with the joule budget, then reassembly and ML-DSA verification. EnergyGate decides on the *first fragment*. DROP is silence; the radio still receives the remaining fragments, and that residual cost is measured, not assumed away.

Score. A depth-4 decision tree, exported to C, maps eight link features over a rolling 5 s window to the probability that the sender is genuine: `hs_per_s`, `hs_fail`, `rssi_mean`, `rssi_var`, `loss_pct`, `dup_pct`, `frag_complete_pct`, `battery_pct`. The shipped model was trained on SIMULATED traces.

Joule budget. A token bucket of capacity B refilling at r J/s; an admitted handshake is charged c_h, a challenge c_c. Every run used the PLACEHOLDER values $B = 20$ J, $r = 0.05$ J/s, $c_h = 0.02$ J, $c_c = 0.5$ mJ. Measured values (Section VII): field-1's share of a handshake is ≈14.9 mJ (so c_h overcharges it by 1.3×), a failed FORGE verify ≈3.5–3.9 mJ (5× overcharge),

TABLE II
PRIMITIVE TIMINGS, RADIO OFF (MS; MEDIAN OF FOUR BOARDS, 5 SESSIONS EACH; SIGNING POOLED, $n = 116$)

Algorithm	keygen	encaps/sign	decaps/verify
ML-KEM-512	5.93	6.05	7.35
ML-KEM-768	8.59	9.88	11.60
ML-KEM-1024	13.14	14.37	16.46
ML-DSA-44	15.36	48.8 (p10 24.1, p90 92.4)	15.97
ML-DSA-65	26.00	63.4 (p10 37.1, p90 152.7)	26.25
X25519†	397.45	403.76	—
ECDSA-P256†	191.16	89.04	202.56

†mbed TLS generic C; not a fair PQC-vs-classical comparison.

and $c_c \lesssim 0.4\,\text{mJ}$ (an upper bound from the noise floor). Since the bucket never spends more than it holds,

$$\sum_{\text{admits in } [0,T]} c \;\le\; B + rT, \qquad \lim_{T\to\infty} \bar{P}_{\text{charged}} \le r, \quad (1)$$

for *any* admission sequence, whatever the score says. It caps what is *charged*; it caps real drain only when c_h is at least the true cost, and it does not cover reception of frames before the gate decides.

Policy v2 (default), with score p, thresholds 0.7/0.4: (1) $p \ge 0.7$ and budget covers $c_h \to$ SPEND; (2) a valid cookie returned and budget covers $c_h \to$ SPEND whatever the score; (3) budget cannot cover one handshake $\to$ DROP; (4) $p \ge 0.4 \to$ CHALLENGE; (5) otherwise CHALLENGE charged c_c, DROP only if c_c cannot be paid. **Policy v1** (ablation) drops at rule 5 and needs $p \ge 0.4$ at rule 2.

Baselines. NONE; RATELIMIT (one admission per sender per 10 s); COOKIE (challenge every sender until it echoes a cookie; no model, no budget).

VI. MEASUREMENT SETUP

Four ESP32-WROOM-32 devkits (ESP32-D0WD-V3, 240 MHz): field-1 (victim), gateway, attacker, monitor. PlatformIO 6.2.0, espressif32 7.1.3. Energy builds strip the demo peripherals. There is no battery: `battery_pct` is fixed at 100 (SIMULATED).

Rig. Field-1 is powered from a 5 V adaptor *only* through a high-side INA219 (0.1 Ω shunt, 0.1 mA/bit), read by the monitor at ≈989 samples/s over 400 kHz I²C. Field-1 raises GPIO4 around each measured operation; the monitor reports per-interval duration, energy and peak current, and a true 1 s mean power.

Calibration. Seven loads from 0 to 232 mA against a DT830D multimeter fit $I_{\text{INA}} = 1.0367\, I_{\text{ref}} - 0.428\,\text{mA}$; after correction residuals are within 0.6 % over 57–232 mA and ≈0.17 mA below 20 mA. Systematics are quoted separately: the 3.67 % gain difference between the two instruments is unresolved and affects absolute joules (C1) but cancels in ratios. Raw logs stay uncorrected; correction is applied in analysis.

Method note. The monitor's instantaneous current sample aliases against a 5 Hz attacker (one FORGE run read ≈0 mW, another +125 mW). All power results use the 1 s mean.

TABLE III
ENERGY PER OPERATION, RADIO OFF (MEAN, $n = 20$). "ABOVE IDLE" SUBTRACTS 225 MW × DURATION

Operation	ms	total mJ	above idle mJ
ML-KEM-768 keygen	8.6	2.91	0.98
ML-KEM-768 encaps	9.7	3.27	1.10
ML-KEM-768 decaps	11.5	3.93	1.34
ML-DSA-44 sign	44	15.4	5.4 ± 1.3
ML-DSA-44 verify	16.2	5.57	1.93
ML-DSA-65 keygen	26.0	9.05	3.19
ML-DSA-65 sign	86	30.0	10.6 ± 4.1
ML-DSA-65 verify	26.2	9.14	3.23
X25519 shared secret†	476	150	43.3
ECDSA-P256 sign†	115	36	9.9

VII. RESULTS: COST OF THE HANDSHAKE (C1)

Deterministic timings agree across all four boards within 0.6 % (Table II); signing is rejection-sampled, so we report its pooled distribution (ML-DSA-65 max 394.8 ms). Table III gives energy per operation (radio off, $n = 20$ per op; idle 225 mW).

At level 3, field-1's share of a handshake is verify + encaps + sign $= 3.23 + 1.10 + 10.6 \approx \mathbf{14.9}$ mJ above idle; both sides together cost ≈31 mJ. On the live mesh with the radio always on (field-1 idles at ≈598 mW), one legitimate handshake costs 9–65 mJ above idle (80–560 ms, dominated by signing variance); one admitted FORGE HELLO ≈3.45 mJ per marked operation; one admitted FLOOD HELLO is not measurably different from zero. Peak crypto current is ≈71 mA. The idle radio dominates absolute drain, so any battery-life claim depends on a duty-cycle assumption we state rather than hide: our victim never sleeps.

VIII. RESULTS: ATTACKS AND DEFENCES (C2, C3)

Each cell of Table IV attacks for 120 s at 5 HELLO/s (SLOW_DRIP: ≈1/min). Extra power is field-1's mean power over the attack minus the minute before; $n = 5$, 95 % CI. Legit success is the gateway's completed/started handshakes during the attack.

Findings. (i) FORGE, not FLOOD, is the cheap-attacker/costly-victim attack: one attacker signature forces a ≈3.5 mJ verify per resend, while random signatures cost nothing measurable. (ii) INSIDER is the largest undefended drain and starves the gateway; every defence removes the drain, but RATELIMIT does so by also locking out the gateway under FLOOD (0 %). (iii) Gate v1 locks out the gateway (2–4 %): forged and real HELLOs share a sender id and look alike to window features, so dropping low scores drops both. Gate v2's cookie rule fixes that. (iv) ADAPTIVE evades the score and still drains +25.5 mW under gate v2. (v) In a rate sweep (5/20/50 s^{-1}, $n = 3$) INSIDER's undefended drain is flat (+94 to +96 mW) because the single crypto slot saturates at 5/s; the gate holds it near +1.5 mW at every rate. A fast FLOOD costs no energy but drops legit success from ≈0.5 to ≈0.07 through airtime and reassembly contention—a different denial than the

TABLE IV
E5 ENERGY GRID: EXTRA VICTIM POWER AND LEGITIMATE SUCCESS

Defence	Attack	Extra power (mW)	Legit success
none	FLOOD	$+1.8 \pm 2.2$	0.71 ± 0.23
	INSIDER	$\mathbf{+91.2 \pm 3.3}$ (+15.2 %)	0.21 ± 0.15
	SLOW_DRIP	$+0.2 \pm 0.6$	0.96 ± 0.11
	FORGE	$\mathbf{+19.0 \pm 1.4}$ (+3.2 %)	0.28 ± 0.24
	ADAPTIVE	$\mathbf{+28.9 \pm 2.3}$	0.30 ± 0.30
	ADAPT._SPOOF	-0.4 ± 1.8	0.65 ± 0.64
ratelimit	FLOOD/INS./DRIP	$-2.2/-1.4/+0.4$	**0.00**/0.87/0.81
cookie	FLOOD/INS./DRIP	$+1.8/+1.4/-0.1$	0.79/0.77/1.00
	FORGE/AD./A.S.	$+0.8/+1.2/-1.4$	0.24/0.53/0.52
gate v2	FLOOD/INS./DRIP	$+1.7/+1.1/+0.7$	0.75/0.82/1.00
	FORGE/A.S.	$+1.4/-0.4$	0.23/0.47
	ADAPTIVE	$\mathbf{+25.5 \pm 3.0}$	0.45 ± 0.33
gate v1	FLOOD/INSIDER	$-2.6/-3.0$	**0.04/0.02**

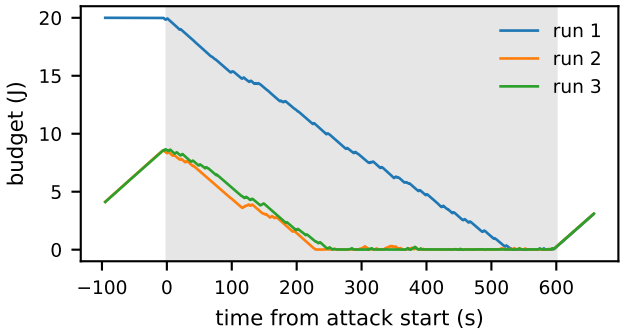


Fig. 1. Field-1's joule budget during three 600 s FORGE+ECHO attacks (shaded) under gate v2. Once empty, admissions are pinned to the refill rate.

energy one. (vi) A 30-min SLOW_DRIP shows no measurable extra power.

FORGE + ECHO defeats the cookie. An attacker echoing the broadcast cookie draws $+15.1 \pm 2.4$ mW under COOKIE and $+16.0 \pm 0.5$ mW under gate v2 ($n = 5$), with ≈540 forged HELLOs spent per run and legit success ≈0.13. The cookie as built is *reusable*: it is bound to neither the HELLO's nonce nor its (epoch, seq), so ≈30 echoes admit every later resend of the cached template, and rule 2 lets them through. This is a real weakness; binding the cookie to the HELLO or making it single-use would close it and is left for a rerun.

The budget binds. We ran FORGE+ECHO for 600 s against gate v2, three times (Fig. 1, Table V; boards on USB, no INA219, so this measures the budget's behaviour, not power). From a full bucket it empties at +537 s (predicted ≈500 s), after which admissions settle at 2.48–2.49 s^{-1}, exactly the refill bound $r/c_h = 0.05/0.02 = 2.5\,s^{-1}$ of (1); the rest are dropped by rule 3. Runs 2–3 started part-filled (8.6 J; 170 s between runs refills only 8.5 J). Because the cap is on what is charged, the real forced work after exhaustion is $\approx 2.5 \times 3.45 \approx 8.6$ mW (ESTIMATED), down from +15–16 mW measured before. The bound held although both the cookie and the score were beaten.

TABLE V
BUDGET RUNS: FORGE+ECHO, 600 S, GATE V2

Run	start	empty at	spend/s before	after	drop/s after
1	20.0 J	+537 s	4.52	**2.48**	2.20
2	8.6 J	+235 s	4.13	**2.49**	1.79
3	8.6 J	+260 s	4.10	**2.49**	2.17

TABLE VI
ATTACK DETECTION, LEAVE-ONE-SESSION-OUT (6 FOLDS). DETECTION RATES AT SCORE THRESHOLD 0.5

Model	AUC	TPR@1%	FPR	DRIP	ADAPT.	A.SP.
carried (SIMULATED)	0.876	0.751	0.000	0.00	0.00	0.00
rate rule	0.905	0.809	0.000	0.00	0.82	0.06
tree, 7 feat.	0.935	0.881	0.014	0.05	0.93	0.74
tree, no RSSI	**0.950**	**0.903**	0.008	0.07	0.97	0.86

IX. ENERGYGATE ON RECORDED TRACES

The shipped tree was trained on SIMULATED traces. We recorded field-1's own eight features and the carried score once a second (6 attacks × 3 sessions plus the 3 budget runs; ≈7 800 rows) and evaluated *leave-one-session-out*—never randomly, since neighbouring rows share a 5 s window. Transition rows are excluded; `battery_pct` is constant and dropped. RSSI is 0 when no HELLO arrived, so it can leak "any HELLO = attack"; we report with and without it (Table VI).

The carried SIMULATED-trained model flags FLOOD, INSIDER, FORGE and FORGE+ECHO perfectly with no false alarms, but misses SLOW_DRIP, ADAPTIVE and ADAPTIVE_SPOOF entirely. Retraining on recorded traces lifts AUC to 0.95 and catches most adaptive traffic; dropping RSSI *helps*, consistent with the leak. SLOW_DRIP stays essentially invisible (7 %) to every model. All sessions come from one rig, one room and one distance, so a held-out session is not a held-out environment. The honest reading is C3's: the classifier is evadable; the budget is the guarantee.

X. LIMITATIONS

- **No battery**: a 5 V adaptor stands in; battery figures are derived from joules, and the victim's radio never sleeps.
- **Placeholder budget**: every run used $c_h = 20$ mJ etc.; measured costs are not yet written back.
- **Attacker energy** is not measured (no second monitor), so the victim-to-attacker energy ratio of C2 is still outstanding; we report victim power only.
- **Reusable cookie** (Section VIII); fix and rerun pending.
- **Absolute energy** rests on an unresolved 3.67 % instrument gain difference; ratios are unaffected.
- **Unauthenticated control channel**, demo keys compiled into images, no formal verification, no level 5, one chip family, one victim and one legitimate peer; legit-success CIs are wide (few handshakes per window).
- **Replay ring**: a captured HELLO whose epoch aged out of the 8-entry ring can be replayed once.

- **Budget run** measured counters, not power; the 8.6 mW post-exhaustion figure is ESTIMATED.

XI. Conclusion

On a battery-powered post-quantum responder the battery, not the message, is the attack surface: an enrolled insider raises the victim's power by 15 % and a single forged signature, resent, forces a full ML-DSA verify every time. A first-fragment gate removes most of the damage cheaply, but a learned score is evadable and a broadcast cookie is echoable. The joule budget is the part that does not depend on being right about the attacker: on the rig it pinned admissions to r/c_h even after the cookie and the score were both beaten. Next steps are a nonce-bound cookie, measured costs written into the budget, a second monitor for attacker energy, and a duty-cycled victim.

Artifact

`firmware/` (all board firmware, host tests), `ml/` (EnergyGate training/export), `paper/data/` (raw logs, never edited; `paper/runlog.csv` lists every run, excluded ones marked), `paper/analysis/` (scripts rebuilding every table, including `e9_retrain.py` and `fig_budget.py`).

References

[1] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, Aug. 2024.

[2] NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, Aug. 2024.

[3] M. J. Kannwischer *et al.*, "Improving software quality in cryptography standardization projects," in *Proc. IEEE EuroS&P Workshops*, 2022. (PQClean.)

[4] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide.

[5] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," SANS ICS / E-ISAC, Mar. 2016.

[6] Gillot, Buchanan, and Tehrani, "Energy consumption of post-quantum cryptography on constrained and general-purpose architectures," *Cryptography*, vol. 10, no. 4, 2026.

[7] Gillot, Buchanan, and Turino, "Testing TLS 1.3 PQC on constrained devices and microcontrollers," arXiv:2610.27766, 2026.

[8] Karcz, Niemiec, Kourtis, and Köppl, "Exploring hardware implementation feasibility of post-quantum cryptography in embedded systems: ML-KEM, ML-DSA and SLH-DSA on ESP32-C6," *PLOS ONE*, 2026.

[9] Lee, Chang, Yu, and Yeh, "On the security implications of PQC in TLS: handshake exhaustion and IDS degradation," arXiv:2607.12504, 2026.

[10] R. Hummen, H. Wirtz, J. H. Ziegeldorf, J. Hiller, and K. Wehrle, "Tailoring end-to-end IP security protocols to the Internet of Things," in *Proc. IEEE ICNP*, 2013.

[11] D. Stebila, L. Kuppusamy, J. Rangasamy, C. Boyd, and J. Gonzalez Nieto, "Stronger difficulty notions for client puzzles and denial-of-service-resistant protocols," in *CT-RSA*, 2011.

[12] Y. Nir and V. Smyslov, "Protecting Internet Key Exchange Protocol Version 2 (IKEv2) implementations from distributed denial-of-service attacks," RFC 8019, 2016.

[13] F. Stajano and R. Anderson, "The resurrecting duckling: security issues for ad-hoc wireless networks," in *Security Protocols Workshop*, 1999.

[14] M. Pirretti *et al.*, "The sleep deprivation attack in sensor networks: analysis and methods of defense," *Int. J. Distrib. Sensor Netw.*, vol. 2, no. 3, pp. 267–287, 2006.